



UNIVERSITÀ DEGLI STUDI DI SALERNO
Dipartimento di Informatica
Corso di Laurea Triennale in Informatica
TESI DI LAUREA TRIENNALE

Ingegnerizzazione di un sistema CRM a microservizi: Integrazione, Automazione dei processi e Business Intelligence

RELATORE

Prof. Fabio Palomba

Università degli Studi di Salerno

TUTOR AZIENDALE

Christian Vitale

Large Systems

CANDIDATA

Sara Di Tella

Matricola: 0512120582

Anno Accademico 2025-2026

«La semplicità è il prerequisito dell'affidabilità.»

Edsger W. Dijkstra (1970)

Abstract

Questo lavoro di tesi documenta l'esperienza di tirocinio curriculare svolta in azienda, dedicata al progettare e implementare una piattaforma CRM a microservizi. Il progetto è nato per superare la frammentazione dei dati commerciali, finora gestiti tramite strumenti isolati e non comunicanti tra loro, che rendeva difficile ricostruire lo stato delle trattative in corso. La soluzione adottata è una piattaforma centralizzata in grado di preservare la riservatezza dei dati aziendali. Il sistema automatizza i flussi di vendita e genera analisi e statistiche aggiornate sullo stato della pipeline commerciale.

L'intervento si è diviso in due fasi distinte. Inizialmente si è lavorato al *refactoring* dei tre microservizi già presenti in azienda: *Personal Data* (uniformazione delle anagrafiche), *Notification* (automazione dell'invio delle notifiche) e *File Manager* (centralizzazione della gestione degli allegati). Nella seconda fase sono stati sviluppati da zero i due moduli centrali dell'architettura. Il *Configurator* è un motore di orchestrazione con vista *Kanban* per gestire l'intero ciclo di vita delle trattative commerciali e registrarne lo storico tramite *Audit Log*. Il modulo *Stats*, dedicato alla *Business Intelligence* elabora invece metriche di vendita quali il tasso di successo delle opportunità (*Win Rate*), le motivazioni di perdita dei contratti e i tempi medi di stazionamento nelle singole fasi della pipeline.

L'architettura si basa su Java e sul *framework* Spring Boot. Le comunicazioni via rete tra i singoli servizi sono affidate a Spring Cloud OpenFeign, mentre la persistenza dei dati si basa su PostgreSQL. L'uso combinato di Spring Data JPA e delle *Criteria API* ha permesso di costruire query con filtri opzionali e composizione dinamica delle condizioni, non ottenibile con query JPQL statiche. La sicurezza degli endpoint è gestita tramite token JWT stateless e un modello di controllo degli accessi basato sui ruoli (RBAC), con il token propagato alle chiamate inter-servizio tramite gli *interceptor* di Feign.

L'isolamento dei microservizi garantisce la continuità delle operazioni principali: un eventuale malfunzionamento del modulo *Stats* o del servizio *Notification*, ad esempio,

non compromette la gestione delle trattative su *Configurator*. L'aver disaccoppiato i servizi semplifica l'integrazione delle funzionalità future, già previste per la gestione della fatturazione e il monitoraggio dei flussi di cassa.

Indice

Elenco delle Figure	iv
Elenco delle Tabelle	v
1 Introduzione e Contesto Aziendale	1
1.1 Contesto Applicativo	1
1.2 Inquadramento del Problema e Motivazioni	2
1.2.1 Criticità di Business e Memoria Organizzativa	2
1.3 L'Approccio Metodologico: <i>Minimum Viable Product</i>	3
1.4 Risultati Ottenuti	3
1.5 Struttura dell'Elaborato	3
2 Background e Stato dell'Arte	5
2.1 Tipologie di Manutenzione del Software	5

2.2	Architettura a Microservizi e Scelte Tecnologiche	6
2.2.1	Stack Applicativo: Java 21 e Spring Boot 3.x	7
2.2.2	Comunicazione Inter-Servizio e Sicurezza	7
2.3	Pattern Architeturali di Codice: DTO, Model, Entity e Mappatura . .	8
2.3.1	Mappatura degli Oggetti e Decisioni Tecnologiche: MapStruct	9
2.4	Persistenza, Versionamento e Containerizzazione	10
2.4.1	Database Relazionale PostgreSQL e Pattern Database-per-Service	11
2.4.2	Versionamento dello Schema Relazionale tramite Flyway . . .	11
2.4.3	Containerizzazione e Gestione degli Ambienti con Docker . .	11
2.4.4	Meccanismo di Property Resolution e Spring Profiles	12
3	Il Sistema Realizzato: Analisi, Architettura e Progettazione	14
3.1	Il Contesto di Partenza: Dominio Applicativo e Applicazione Legacy	15
3.1.1	Da <i>MedData</i> a <i>crm-personaldata</i> : il Punto di Partenza . . .	15
3.1.2	Il Dominio Applicativo e i Requisiti Ereditati	16
3.1.3	Integrazione Architeturale e Isolamento dei Dati	17
3.2	Ingegneria dei Requisiti e Analisi del Dominio	17
3.2.1	Requisiti Funzionali	17
3.2.2	Requisiti Non Funzionali e Definizione dei Design Goals . . .	20
3.3	L'Architettura del Sistema CRM	21
3.3.1	Topologia dei Microservizi e Componenti di Dominio	22
3.3.2	Comunicazione Inter-Servizio: OpenFeign e Configurazione Dinamica	24

3.4	Reingegnerizzazione dei Moduli Legacy	24
3.4.1	Modulo <code>crm-personaldata</code> (Anagrafica Centralizzata) . .	24
3.4.2	Modulo <code>crm-notification</code>	27
3.4.3	Modulo <code>crm-filemanager</code>	27
3.5	Sviluppo del Modulo <code>crm-configurator</code>	27
3.5.1	Macchina a Stati e Logiche di Business (Opportunity)	28
3.5.2	CQRS, Motore di Ricerca Dinamico e Paginazione	30
3.5.3	Tracciabilità Storica (Event Sourcing)	31
3.6	Sviluppo del Modulo <code>crm-stats</code> (Business Intelligence)	31
3.6.1	Integrazione OpenFeign e Propagazione della Sicurezza . . .	31
3.6.2	Algoritmi Analitici di Business Intelligence	31
4	Validazione del Sistema, Testing e Sviluppi Futuri	33
4.1	Fondamenti Teorici del Testing e Strategia Adottata	34
4.2	Validazione Interna	34
4.3	Troubleshooting Ingegneristico e Risoluzione delle Criticità	35
4.4	Validazione Esterna e User Acceptance Testing (UAT)	36
4.5	Sviluppi Futuri ed Evoluzione del Prodotto	37
5	Conclusioni	38
5.1	Bilancio del Progetto e Obiettivi Raggiunti	38
5.2	Valutazione Critica e Competenze Acquisite	39
	Bibliografia	41

Elenco delle figure

2.1	Flusso di conversione dei dati tramite il pattern DTO e mappatura attraverso i livelli architetturali.	9
3.1	Rappresentazione ad alto livello dell'architettura a microservizi della piattaforma CRM sviluppata.	22
3.2	Schema E-R Logico-Fisico del modulo <code>crm-personaldata</code> a seguito della reingegnerizzazione.	25
3.3	Schema E-R Logico-Fisico del modulo <code>crm-configurator</code> : architettura relazionale per la gestione delle trattative, catalogo prodotti e Audit Log.	28
3.4	Statechart Diagram: Macchina a stati finiti della trattativa commerciale per la logica Kanban.	29
3.5	Interfaccia utente: Struttura della <i>Kanban Board</i> mappata sugli stati della pipeline commerciale.	30

Elenco delle tabelle

3.1	Fasi operative di setup e allineamento del modulo anagrafiche legacy.	16
3.2	Scheda Use Case — Avanzamento Pipeline Kanban.	18
3.3	Interventi principali di refactoring sul modulo <code>crm-personaldata</code> .	26
4.1	Principali criticità emerse in fase di troubleshooting e relative soluzioni.	35

CAPITOLO 1

Introduzione e Contesto Aziendale

Questo elaborato documenta lo sviluppo software condotto durante le 425 ore di tirocinio formativo curriculare. Il fulcro del progetto ha riguardato la realizzazione di un sistema CRM (*Customer Relationship Management*) basato su un'architettura distribuita, finalizzato a ottimizzare i processi commerciali aziendali e a riorganizzare in modo strutturato la gestione dei dati.

Il capitolo fornisce una panoramica del contesto applicativo: vengono presentate le criticità operative all'origine dell'intervento, le motivazioni che hanno guidato le scelte progettuali e l'approccio metodologico adottato per conseguire i risultati descritti nel resto dell'elaborato.

1.1 Contesto Applicativo

L'esperienza di tirocinio si è svolta presso la sede di **Large Systems**, una realtà IT specializzata nello sviluppo di software enterprise e nell'erogazione di servizi appli-

cativi per il settore B2B. L'azienda è orientata alla migrazione da sistemi monolitici verso architetture a microservizi e cloud.

L'organizzazione dello sviluppo si basa su metodologie *Agile/Scrum*, affiancate da pratiche *DevOps* per l'integrazione e il rilascio continuo (CI/CD) [1]. La collaborazione all'interno del team è supportata da strumenti di tracciamento delle attività (*issue tracking*) e da flussi di controllo versione strutturati.

1.2 Inquadramento del Problema e Motivazioni

Il progetto nasce dall'osservazione di alcune inefficienze concrete nella gestione dei flussi commerciali interni all'azienda.

1.2.1 Criticità di Business e Memoria Organizzativa

Prima dell'introduzione della nuova piattaforma CRM, la conoscenza relativa ai clienti, ai referenti e allo stato di avanzamento delle trattative commerciali era frammentata e dispersa tra caselle di posta elettronica, fogli di calcolo condivisi e note personali. Questa condizione esponeva l'azienda al rischio sistematico di dispersione della memoria organizzativa, specialmente durante avvicendamenti interni, assenze o passaggi di consegne nel team di vendita. L'assenza di un archivio unico e centralizzato determinava frequenti inconsistenze nei dati, quali duplicazioni di record, schede anagrafiche incomplete ed errori manuali dovuti a sovrascritture concorrenti non tracciate. Di riflesso, la gestione commerciale operava senza visibilità analitica: il management si trovava impossibilitato a monitorare l'evoluzione della pipeline e a estrarre indicatori di prestazione affidabili, come il tasso di conversione (*Win Rate*) o le cause ricorrenti di perdita delle opportunità.

L'obiettivo del progetto è stato quindi trasformare un processo commerciale non tracciato in un flusso di lavoro centralizzato, misurabile e supportato da automatismi di processo.

1.3 L'Approccio Metodologico: *Minimum Viable Product*

Dato il vincolo temporale delle 425 ore di tirocinio, la pianificazione ha escluso fin da subito la realizzazione di un prodotto finito completo di ogni funzionalità accessoria, orientando lo sviluppo verso il rilascio di un **Minimum Viable Product (MVP)** [2].

L'approccio MVP mira a mitigare i rischi di progetto, consentendo di validare tempestivamente i requisiti e di rendere disponibile valore operativo già dalle prime iterazioni. Lavorando a stretto contatto con la *Project Manager*, referente per i requisiti e *stakeholder* principale, lo sviluppo si è concentrato sul nucleo centrale (*Core Domain*) dell'applicativo: l'anagrafica unificata, il motore di orchestrazione delle trattative commerciali e la reportistica di base.

1.4 Risultati Ottenuti

Al termine delle attività, il sistema CRM è stato consolidato e validato internamente, rispondendo ai requisiti definiti in fase di analisi. L'intervento ha prodotto due risultati principali, approfonditi nei capitoli successivi:

1. Il riadattamento, la modernizzazione e la stabilizzazione di moduli software *legacy* preesistenti (gestione anagrafiche, storage di documenti e notifiche), resi conformi ai nuovi standard aziendali e predisposti all'integrazione.
2. La progettazione e lo sviluppo *ex-novo* di un motore logico per la gestione visiva delle trattative e di un modulo analitico per l'elaborazione dei dati di *Business Intelligence*.

1.5 Struttura dell'Elaborato

La tesi è strutturata nei seguenti capitoli:

- Il **Capitolo 2** introduce il quadro teorico e tecnologico di riferimento: illustra i principi *DevOps* e le pipeline CI/CD, la containerizzazione mediante Docker e i principali design pattern per architetture distribuite a microservizi, offrendo una trattazione generale e indipendente dal contesto applicativo specifico.
- Il **Capitolo 3** descrive la progettazione e l'implementazione del sistema: parte dall'analisi dei requisiti e dalla definizione dell'architettura per Large Systems, illustra gli interventi di manutenzione adattiva sui moduli ereditati e approfondisce lo sviluppo dei nuovi componenti per il configuratore commerciale e la *Business Intelligence*.
- Il **Capitolo 4** espone le attività di verifica e collaudo: analizza i test funzionali, il *troubleshooting* delle anomalie emerse in fase di integrazione, gli esiti dello *User Acceptance Testing* (UAT) con gli stakeholder e le prospettive di sviluppo futuro del sistema.
- Il **Capitolo 5** raccoglie le considerazioni conclusive, tracciando un bilancio dell'esperienza formativa e dei risultati raggiunti.

CAPITOLO 2

Background e Stato dell'Arte

Qui si delineano i fondamenti architetturali su cui si fonda il lavoro descritto nei capitoli successivi. La prima sezione inquadra la manutenzione del software dal punto di vista ingegneristico, classificando gli interventi secondo le categorie previste dalla letteratura; le sezioni successive descrivono invece l'architettura a microservizi, i *design pattern* e le tecnologie di persistenza e containerizzazione adottate nel progetto.

2.1 Tipologie di Manutenzione del Software

Un sistema software, una volta rilasciato, continua a evolversi e ad adattarsi per tutta la sua vita utile. In tale contesto si collocano i processi di manutenzione del software. Sommerville [3] distingue le attività manutentive in due macro-categorie.

La **Manutenzione Adattiva** (*Adaptive Maintenance*) riguarda gli interventi necessari ad adeguare un sistema preesistente a cambiamenti dell'ambiente operativo o infrastrutturale: l'aggiornamento dello stack applicativo a una nuova versione del

framework, la migrazione tra standard tecnologici (come la transizione da *Java EE* a *Jakarta EE*), o l'adeguamento di moduli applicativi preesistenti a nuove specifiche di linguaggio o di framework.

Invece, la **Manutenzione Perfettiva** (*Perfective Maintenance*) comprende gli interventi proattivi volti a introdurre requisiti di business non previsti nelle specifiche originali, o a migliorare prestazioni e manutenibilità del sistema nel suo complesso. Rientra in questa categoria la progettazione e lo sviluppo *ex-novo* di componenti non presenti nel sistema originario, pensati per introdurre nuove funzionalità di business o per ampliare le capacità analitiche del sistema.

In questo elaborato di tesi sono state attraversate entrambe le categorie: la reingegnerizzazione dei moduli ereditati per l'anagrafica clienti (`crm-personaldata`), le notifiche (`crm-notifications`) e la gestione documentale (`crm-filemanager`) rientra nella manutenzione adattiva, mentre lo sviluppo *ex-novo* del motore di gestione delle trattative (`crm-configurator`) e del modulo di Business Intelligence (`crm-stats`) è manutenzione perfetta.

2.2 Architettura a Microservizi e Scelte Tecnologiche

Un'alternativa sempre più diffusa al classico approccio monolitico è la topologia a **microservizi** autonomi e debolmente accoppiati (*loosely coupled*) [4]. L'adozione di questo paradigma è motivata da benefici strutturali emersi in fase di analisi dei requisiti, come isolare i singoli domini applicativi, contenere la propagazione dei guasti (*fault isolation*) e permettere la scalabilità orizzontale indipendente ai soli servizi effettivamente soggetti a carichi elevati [5].

In un'architettura di questo tipo, l'infrastruttura viene tipicamente organizzata in moduli indipendenti, ciascuno responsabile di un dominio di business ben preciso: un punto di ingresso unico che smista il traffico verso i servizi corretti (*API Gateway*), un modulo dedicato alla gestione delle identità e delle credenziali, uno o più moduli di dominio che incapsulano la logica di business specifica, ed eventuali moduli

satellite per funzioni trasversali come l’invio di notifiche o la gestione dei documenti binari. La traduzione applicativa di questo modello è descritta nel Capitolo 3.

2.2.1 Stack Applicativo: Java 21 e Spring Boot 3.x

Uno standard di riferimento per lo sviluppo di sistemi enterprise a microservizi è costituito da **Java 21** in combinazione con il framework **Spring Boot 3.x** [6]. L’adozione di questa versione del framework impone il passaggio obbligato dallo standard *Java EE* alla specifica **Jakarta EE**: un’operazione di *refactoring* che richiede di sostituire tutte le dipendenze di persistenza (`javax.persistence.*`) e di validazione (`javax.validation.*`) con i corrispondenti pacchetti `jakarta.*`.

Questo passaggio è stato uno dei primi interventi condotti sul modulo dell’anagrafica clienti (`crm-personaldata`), come descritto nella Tabella 3.1.

2.2.2 Comunicazione Inter-Servizio e Sicurezza

Nelle architetture distribuite, sicurezza e comunicazione di rete sono strutturate su due piani distinti.

A livello perimetrale, le richieste del client vengono intercettate da un API Gateway, che funge da *Reverse Proxy* e da *Policy Enforcement Point* (PEP) [5]. Il modello di autenticazione adotta token **JWT (JSON Web Token)** *stateless*: una volta validata la firma a livello di Gateway, i ruoli dell’utente vengono iniettati negli header HTTP, così da poter applicare il controllo d’accesso basato sui ruoli (RBAC) nei servizi a valle.

La comunicazione interna tra microservizi invece si affida a client dichiarativi come **Spring Cloud OpenFeign** [7]. Gli endpoint dei client vengono configurati con proprietà dinamiche provviste di valori di default, risolti a runtime in base all’ambiente di esecuzione (sviluppo locale o container). Tale disaccoppiamento previene errori di risoluzione durante le fasi di deployment, le cui implicazioni pratiche sono approfondite nel Capitolo 3.

2.3 Pattern Architetture di Codice: DTO, Model, Entity e Mappatura

Separare la rappresentazione persistente del dato dalla sua esposizione verso l'esterno, secondo il principio di *Separation of Concerns* [3], necessita di quattro tipologie di oggetti, ciascuna confinata al proprio livello architetturale: è questo il modo più diretto per prevenire quello che in letteratura viene chiamato *Entity Leakage*.

Il **Request DTO** (*Data Transfer Object*) è collocato nel *Presentation Layer*, in ingresso: trasporta il *payload* JSON inviato dal client e rappresenta la prima barriera di sicurezza del sistema, grazie alle annotazioni di validazione formale *JSR-380/Jakarta Validation* (@NotNull, @NotBlank, e simili). Serve a prevenire vulnerabilità come il *Mass Assignment*, impedendo che vengano manipolati campi interni non autorizzati. Sul lato opposto c'è il **Response DTO**, anch'esso nel *Presentation Layer* ma in uscita: struttura il *payload* restituito al client seguendo il principio dell'*Information Hiding*, in modo da non esporre la struttura interna del database relazionale.

Nello strato intermedio si colloca il **Model** (Domain Model), l'oggetto di business usato esclusivamente nello strato di servizio (*Business Logic Layer*). Risulta completamente disaccoppiato sia dai protocolli di trasporto (HTTP) sia dai dettagli di persistenza: serve solo a eseguire la logica applicativa pura e a trasportare in modo sicuro le informazioni tra un microservizio e l'altro.

Invece, l'**Entity** (JPA Entity) appartiene al *Data Persistence Layer* ed è mappata in modo esatto sulle tabelle del database relazionale tramite *Object-Relational Mapping* (ORM), impiegata esclusivamente dai *Repository* per leggere e scrivere sul database.

La Figura 2.1 mostra il flusso di trasformazione dei dati lungo questi quattro strati. Il passaggio da un livello all'altro non è manuale, ma affidato a un framework di mappatura, *MapStruct*, che, lavorando a tempo di compilazione, garantisce sicurezza dei tipi (*Type Safety*) e azzerava l'*overhead* prestazionale tipico della *reflection* a runtime.

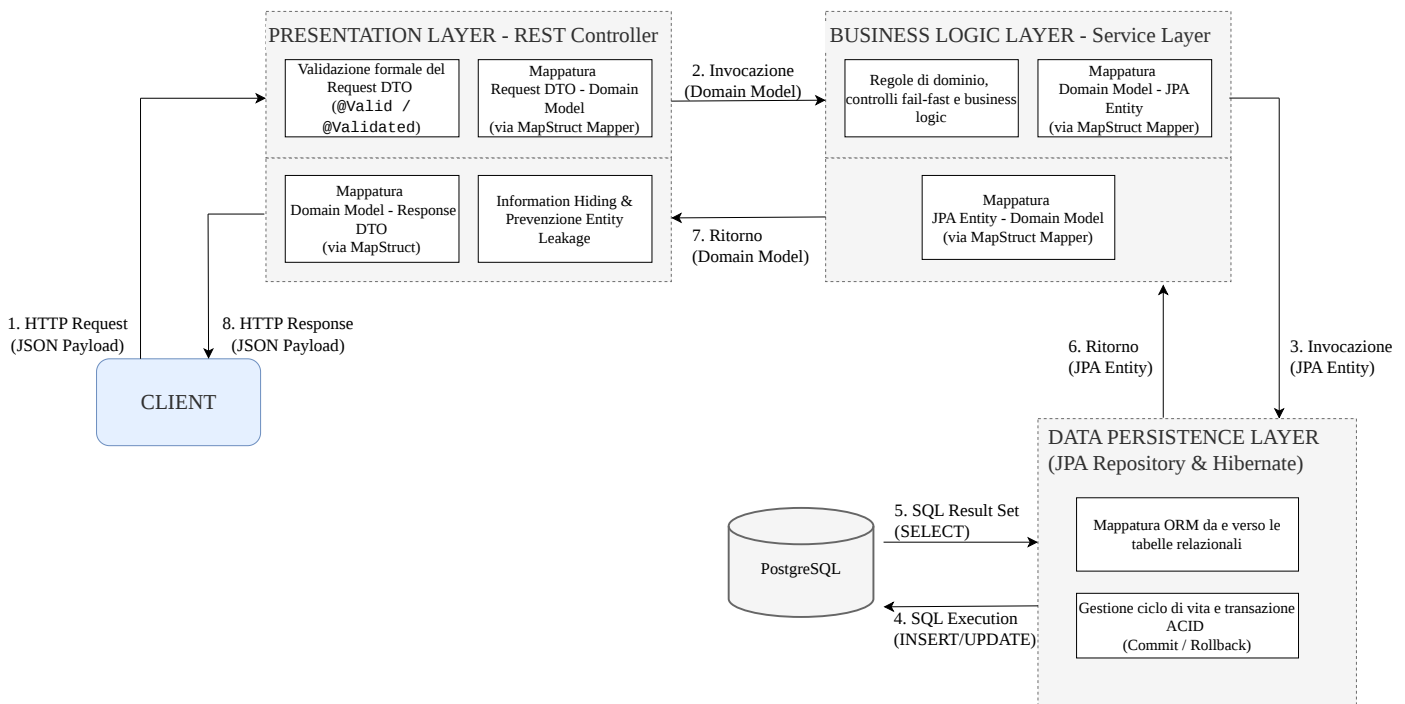


Figura 2.1: Flusso di conversione dei dati tramite il pattern DTO e mappatura attraverso i livelli architetturali.

2.3.1 Mappatura degli Oggetti e Decisioni Tecnologiche: MapStruct

Il progetto adottava inizialmente **ModelMapper** per la conversione tra gli oggetti dei vari layer. Successivamente, la trasformazione dei dati tra i diversi strati architetturali è stata affidata al framework **MapStruct** [8], impiegato come *Annotation Processor* in fase di compilazione (`javac`). A differenza delle librerie di conversione basate su *reflection* a runtime, MapStruct genera codice sorgente Java esplicito composto da invocazioni dirette di getter e setter: questo approccio garantisce la sicurezza dei tipi (*Type Safety*), massimizza le prestazioni di esecuzione e azzerava il consumo computazionale tipico dell'ispezione dinamica.

L'adozione di questa soluzione ha consentito di superare le criticità riscontrate con **ModelMapper**, inizialmente valutato nelle prime iterazioni. L'ispezione a runtime di quest'ultimo si è rivelata particolarmente insidiosa durante le modifiche di refactoring: anche impostando strategie di corrispondenza rigorose (`MatchingStrategies.STRICT`), un disallineamento nei nomi dei campi tra DTO

ed Entity non produceva errori in fase di *build*, ma si traduceva in proprietà impostate a `null` a runtime. Tali anomalie silenziose si propagavano lungo il layer di persistenza, emergendo solo tardivamente sotto forma di violazioni di vincoli a database.

Per prevenire sistematicamente questo problema, la configurazione globale di MapStruct è stata vincolata alla direttiva:

```
unmappedTargetPolicy = ReportingPolicy.ERROR
```

In questo modo, qualsiasi omissione o disallineamento tra i campi dell'oggetto sorgente e di destinazione causa l'immediato fallimento della compilazione via Maven. Questo approccio *fail-fast* sposta la validazione della coerenza dei contratti di mappatura a tempo di sviluppo.

Infine, per la gestione delle operazioni di aggiornamento parziale (PATCH), è stata adottata la policy:

```
nullValuePropertyMappingStrategy =  
NullValuePropertyMappingStrategy.IGNORE
```

che consente a MapStruct di ignorare automaticamente i campi nulli nel *payload* di richiesta, preservando lo stato preesistente dell'entità senza richiedere proliferazioni di controlli condizionali all'interno dei servizi.

2.4 Persistenza, Versionamento e Containerizzazione

Il livello di persistenza relazionale e l'infrastruttura di virtualizzazione locale vengono in genere pensati per un obiettivo preciso: il massimo isolamento e la piena riproducibilità degli ambienti di esecuzione, tra sviluppo locale, collaudo (*development*) e produzione.

2.4.1 Database Relazionale PostgreSQL e Pattern Database-per-Service

Un motore relazionale comune per questo tipo di infrastrutture è **PostgreSQL** [9]. Seguendo il pattern *Database-per-Service*, ogni microservizio ha il proprio database isolato e indipendente: in un'architettura di questo genere, ad esempio, il servizio anagrafico e il servizio operativo non condividono lo strato di persistenza [4].

Questa scelta ha una conseguenza diretta: nessun modulo può fare interrogazioni dirette o *JOIN* inter-database. La comunicazione passa obbligatoriamente per le API REST, e i dati restano rigidamente sotto la responsabilità del rispettivo servizio di dominio.

2.4.2 Versionamento dello Schema Relazionale tramite Flyway

Il controllo evolutivo degli schemi relazionali è demandato a **Flyway** [10], integrato direttamente nel ciclo di avvio di Spring Boot. All'inizializzazione del contesto applicativo, il motore interroga la tabella di metadati `flyway_schema_history` per verificare lo stato di avanzamento delle migrazioni ed esegue, in modo strettamente sequenziale e all'interno di blocchi transazionali isolati, gli script SQL versionati (quali `V1__Create_Tables.sql`) individuati nel *classpath*.

Questo meccanismo garantisce l'allineamento deterministico tra il modello a oggetti definito nel codice sorgente e la struttura fisica del database, azzerando le discrepanze di schema sia durante lo sviluppo concorrente nel team, sia lungo la pipeline di rilascio negli ambienti di collaudo e produzione.

2.4.3 Containerizzazione e Gestione degli Ambienti con Docker

Per lo sviluppo e il collaudo locale, un ecosistema di questo tipo viene tipicamente containerizzato con **Docker** [11]. A differenza delle macchine virtuali tradizionali, che emulano l'intero hardware e introducono un *overhead* prestazionale significativo,

i container Docker condividono il kernel del sistema operativo ospite: isolano i processi a livello utente tramite *namespaces* e *cgroups*, restando estremamente leggeri e reattivi [11].

Questo permette, tra le altre cose, di avviare istanze isolate di un DBMS direttamente sul PC di sviluppo, senza dover installare nulla globalmente e confinando in partenza eventuali conflitti di porte o versioni. Un meccanismo chiave, in questo senso, è il **Port Mapping**, che associa la porta fisica dell'interfaccia di rete dell'host alla porta interna virtuale del container [11]. Quando più ambienti (ad esempio sviluppo, demo e produzione) convivono sulla stessa macchina fisica, per tenerli separati si mappa tipicamente l'host esterno su porte differenziate, lasciando invariata la porta standard all'interno del container.

2.4.4 Meccanismo di Property Resolution e Spring Profiles

Docker e Spring Boot si incontrano, nella pratica, nel meccanismo dinamico di **Property Resolution** [6]. Scrivere in modo rigido (*hardcoding*) le credenziali di accesso o gli URL dei servizi partner direttamente nel codice sorgente è una pratica rischiosa, oltre che una vulnerabilità di sicurezza vera e propria; per questo un'applicazione ben progettata si appoggia invece all'ordine di precedenza delle sorgenti di configurazione previsto da Spring Boot.

Le proprietà definite nel file `application.properties`, dentro il pacchetto `.jar`, servono solo da *fallback* per lo sviluppo in locale. Le variabili d'ambiente iniettate nel container Docker all'avvio hanno invece la precedenza assoluta, e sovrascrivono qualunque valore predefinito. Il meccanismo è tipicamente implementato con una sintassi di espressione del tipo:

```
spring.datasource.url=
jdbc:postgresql://${DB_HOST:127.0.0.1}
:${DB_PORT:5432}/${DB_NAME}
```

Sul PC locale, non trovando variabili d'ambiente, il framework risolve la stringa

sul *loopback* (127.0.0.1:5432). Nel container Docker remoto, sul server, succede l'opposto: l'infrastruttura inietta l'indirizzo privato interno alla rete virtuale Docker bridge, e a tradurlo nel corrispondente indirizzo IP privato ci pensa direttamente il DNS server integrato di Docker, completando così il disaccoppiamento architetturale tra i moduli.

CAPITOLO 3

Il Sistema Realizzato: Analisi, Architettura e Progettazione

La realizzazione della piattaforma CRM ha richiesto l'adeguamento di un'architettura preesistente e lo sviluppo da zero dei servizi dedicati alla gestione commerciale e all'analisi dei dati. Le scelte progettuali adottate contestualizzano i principi presentati nel Capitolo 2 al dominio applicativo aziendale.

L'attività si è articolata in una prima parte che ripercorre gli interventi di reingegnerizzazione condotti sui moduli *legacy* ereditati da un progetto già esistente in azienda (`crm-personaldata`, `crm-notifications`, `crm-filemanager`) e in una seconda che descrive la progettazione e lo sviluppo *ex-novo* dei due microservizi realizzati durante il tirocinio, `crm-configurator` e `crm-stats`.

Le attività si sono svolte seguendo la metodologia *Agile*, orientando le prime iterazioni al rilascio rapido di un *Minimum Viable Product* (MVP) capace di supportare fin da subito l'operatività del team di vendita, e validato di volta in volta con la *Project Manager*, la quale ha ricoperto il doppio ruolo di committente e principale *stakeholder*

del sistema.

3.1 Il Contesto di Partenza: Dominio Applicativo e Applicazione Legacy

L'architettura del nuovo sistema CRM si è innestata su un'infrastruttura aziendale preesistente, ereditando un modulo anagrafico che ha richiesto una marcata attività di scomposizione e riprogettazione.

3.1.1 Da *MedData* a *crm-personaldata*: il Punto di Partenza

Il nucleo di partenza per la gestione delle anagrafiche del CRM era il microservizio *meddata-personaldata*, sviluppato in origine dall'azienda per una piattaforma di dominio completamente diverso, quello medico (*MedData*).

Riscrivere tutto da zero sarebbe stato inefficiente e rischioso in un contesto aziendale reale, per cui si è optato per il riuso del codice esistente: si è clonato il branch stabile *master* del modulo originario, rimosso il puntatore al repository remoto di *MedData* e agganciata la cronologia al nuovo repository di produzione.

Il mantenimento della cronologia Git ha garantito la tracciabilità delle modifiche pregresse, semplificando la diagnosi e la risoluzione delle anomalie nel codice ereditato. La Tabella 3.1 riassume le fasi operative di questo allineamento iniziale.

Tabella 3.1: Fasi operative di setup e allineamento del modulo anagrafiche legacy.

Fase Operativa	Attività Ingegneristica Svolta
Fork di Versione	Clonazione del codice <i>MedData</i> con mantenimento dello storico Git.
Refactoring Naming	Sostituzione globale di ogni pacchetto e riferimento da <i>meddata</i> a <i>crm</i> .
Allineamento POM	Migrazione a Java 21 e Spring Boot 3.x , risolvendo errori di compilazione nel passaggio da <i>javax</i> a <i>jakarta</i> .
DDL Migration	Cancellazione delle vecchie tabelle e scrittura del file Flyway <code>V1__create_tables.sql</code> .

3.1.2 Il Dominio Applicativo e i Requisiti Ereditati

Il dominio del modulo ereditato ruotava attorno all'anagrafica dei **Clienti** (aziende, con ragione sociale, dimensione, settore merceologico e contratti in essere) e dei relativi **Recapiti**, divisi in indirizzi fisici e contatti telematici come e-mail, PEC e telefono; su queste due entità l'applicazione supportava già le operazioni CRUD di base. Mancava del tutto, invece, la gestione dei **Referenti**, ovvero le persone fisiche che lavorano presso i clienti, ciascuna con un ruolo specifico: nel dominio originario di *MedData* non c'era motivo di modellare per nome i singoli contatti umani legati a un'azienda cliente, per cui questa entità è stata introdotta ex-novo nell'ambito del progetto CRM.

Oltre a questa lacuna, l'applicazione presentava due limiti a livello di requisiti di business, che sarebbero poi diventati il punto di partenza degli interventi descritti nella Sezione 3.4. Mancavano, anzitutto, i dati fiscali: non c'erano campi per Partita IVA e Codice SDI, il che rendeva impossibile automatizzare la gestione dei preventivi e l'interfacciamento con il Sistema di Interscambio. C'era poi un problema più strutturale nella classificazione commerciale delle aziende, basata su un Enum a stato

singolo e mutuamente esclusivo: un'azienda non poteva essere allo stesso tempo cliente attivo e partner commerciale abilitato a ricevere royalty sulle vendite, il che costringeva gli utenti a duplicare manualmente i record.

3.1.3 Integrazione Architettuale e Isolamento dei Dati

Per allineare il codice ereditato all'infrastruttura a microservizi descritta nel Capitolo 2, il modulo è stato ricondotto al pattern **Database-per-Service** [4]: un database PostgreSQL isolato, raggiungibile dagli altri servizi solo tramite API REST, garantendo il completo disaccoppiamento dello schema relazionale e prevenendo accessi diretti non mediati da parte degli altri componenti del sistema.

3.2 Ingegneria dei Requisiti e Analisi del Dominio

La fase iniziale delle attività ha previsto un confronto continuo con la Project Manager, attraverso sessioni di analisi e interviste strutturate. Seguendo la classica distinzione dell'Ingegneria del Software [3], i requisiti raccolti sono stati formalizzati in due gruppi: funzionali e non funzionali.

3.2.1 Requisiti Funzionali

L'obiettivo primario del sistema è coprire senza soluzione di continuità l'intero ciclo di vita del processo di vendita. Sulla base delle criticità emerse, l'analisi dei requisiti funzionali ha prioritizzato la centralizzazione del patrimonio informativo e la tracciabilità delle operazioni commerciali. Il nucleo operativo del sistema ruota attorno all'anagrafica unificata di clienti e referenti, progettata per attribuire dinamicamente ruoli multipli (quali cliente finale, azienda partner o entrambi) ed eliminare alla radice la ridondanza dei record. A questa base si aggancia il motore per la gestione visiva della pipeline di vendita: le opportunità commerciali (*Opportunity*) vengono modellate mediante una macchina a stati finiti e governate dal team attraverso una

Kanban Board interattiva, che vincola le transizioni di stato e memorizza lo storico dei cambi di fase.

A completamento dell'infrastruttura applicativa, il modulo di *Business Intelligence* trasforma i dati transazionali in metriche decisionali accessibili in tempo reale. Questo componente computazionale elabora in modo aggregato i flussi di vendita, calcolando indicatori sintetici quali il tasso di conversione (*Win Rate*), la distribuzione del volume d'affari su base temporale e il tempo medio di permanenza delle trattative nei singoli stadi della pipeline, fornendo così al management la visibilità analitica necessaria per identificare tempestivamente colli di bottiglia o cause ricorrenti di insuccesso.

La Tabella 3.2 riporta la caratterizzazione formale del caso d'uso primario per la gestione dell'avanzamento delle trattative commerciali.

Tabella 3.2: Scheda Use Case — Avanzamento Pipeline Kanban.

Nome del Caso d'Uso	Spostare un'opportunità commerciale nella Pipeline
Descrizione	Consente al Commerciale di modificare lo stato di un'opportunità commerciale all'interno della vista Kanban, verificando i vincoli di transizione e registrandone lo storico.
Attori Principali	Commerciale - rappresenta l'operatore di vendita interessato ad avanzare le trattative
Entry Condition	Il Commerciale è autenticato, è abilitato alla gestione delle vendite e visualizza la vista Kanban delle opportunità.

Exit Condition - Success	Lo stato dell'opportunità viene aggiornato nel sistema, viene registrato un record immutabile nello storico delle attività e la probabilità di chiusura viene ricalcolata. La vista Kanban mostra la nuova posizione dell'opportunità.
Exit Condition - Failure	Lo stato dell'opportunità e lo storico rimangono invariati. Il sistema mostra un messaggio informativo sull'errore e riposiziona l'opportunità nello stato di partenza.
Flusso Principale	1. Il Commerciale richiede lo spostamento di un'opportunità commerciale verso un nuovo stato della pipeline. 2. Il sistema verifica che lo spostamento rispetti le regole di transizione previste dalla macchina a stati dell'opportunità. 3. Il sistema aggiorna lo stato dell'opportunità commerciale. 4. Il sistema registra l'evento nello storico delle attività per fini di tracciamento. 5. Il sistema ricalcola la probabilità di chiusura associata al nuovo stato. 6. Il sistema mostra al Commerciale la vista Kanban aggiornata.

Flussi Alternativi	Transizione verso stato di perdita (KO) (passo 2): Se il nuovo stato selezionato indica la perdita dell'opportunità, il sistema verifica se è stato specificato il motivo del KO. In assenza di tale motivo, il sistema richiede al Commerciale di selezionare la causa della perdita. Una volta inserita la causa, lo use case prosegue al passo 3 del flusso principale. Se il Commerciale annulla l'inserimento, si attiva il flusso di errore.
Flussi di Errore	Transizione non valida o annullata (passo 2): Il sistema rileva che le regole di transizione non sono rispettate o che l'inserimento delle informazioni obbligatorie è stato annullato. Il sistema mostra un messaggio di errore, annulla qualsiasi modifica e riposiziona l'opportunità nello stato originale. Lo use case termina con un fallimento.

3.2.2 Requisiti Non Funzionali e Definizione dei Design Goals

Durante le sessioni di analisi con i referenti di progetto, oltre alle funzionalità core sono emersi anche i requisiti qualitativi e i vincoli operativi che il sistema avrebbe dovuto rispettare una volta in produzione: i cosiddetti Requisiti Non Funzionali.

Per tradurli in scelte architetture concrete, ciascun requisito è stato associato, in fase di progettazione, a un corrispondente *Design Goal* (obiettivo di progettazione), secondo l'impostazione teorica dell'Ingegneria del Software [5]. Durante la fase di analisi con la *Project Manager*, l'attenzione si è concentrata su priorità operative che hanno dettato i *Design Goals* del sistema:

Un primo vincolo riguardava **sicurezza e controllo degli accessi**: solo gli utenti autorizzati potevano accedere ai dati commerciali, con permessi differenziati per ruolo. Il

design goal corrispondente è la *Security*, e la risposta architetture, l'autenticazione *stateless* via JWT descritta nel Capitolo 2, nasce proprio per soddisfare questa esigenza specifica.

Sul fronte **usabilità** la *Kanban Board* doveva restare fluida durante lo spostamento delle card tra le fasi. Qui i design goal previsti sono *Usability* e *Low Latency*, con uno spostamento delle card via drag-and-drop con chiamate asincrone lato client, che aggiornano subito i dati a schermo mentre la persistenza avviene in background.

Un errore di rete o un salvataggio parziale non doveva mai lasciare un'opportunità in uno stato incoerente o privo di traccia storica. Proprio per questo il terzo requisito è quello di **integrità e consistenza dei dati**. Il design goal corrispondente è la *Dependability*, garantito da transazioni **ACID** rigorose e da una validazione preliminare lato applicativo, così che ogni transizione di stato risultasse atomica.

Infine la necessità di garantire **isolamento dei guasti e la manutenibilità**, ha orientato la scelta verso un'architettura a microservizi containerizzata, impedendo che un eventuale malfunzionamento nel calcolo dei KPI comprometta l'operatività delle vendite.

3.3 L'Architettura del Sistema CRM

Il sistema sviluppato per Large Systems adotta la stessa logica a microservizi introdotta a livello generale nella Sezione 2.2: componenti autonomi e debolmente accoppiati, pensati per isolare i domini applicativi e contenere la propagazione dei guasti. Nel caso specifico del CRM, questa scelta è emersa direttamente dall'analisi dei requisiti descritta nella Sezione 3.2: separare l'anagrafica clienti dalla logica di vendita, e questa a sua volta dal calcolo degli indicatori di business, in modo che ciascun dominio potesse evolvere ed essere scalato in autonomia.

3.3.1 Topologia dei Microservizi e Componenti di Dominio

La Figura 3.1 mostra come l'infrastruttura sia composta da sette moduli indipendenti, ciascuno responsabile di un dominio di business ben preciso. Di seguito vengono ripercorsi uno per uno.

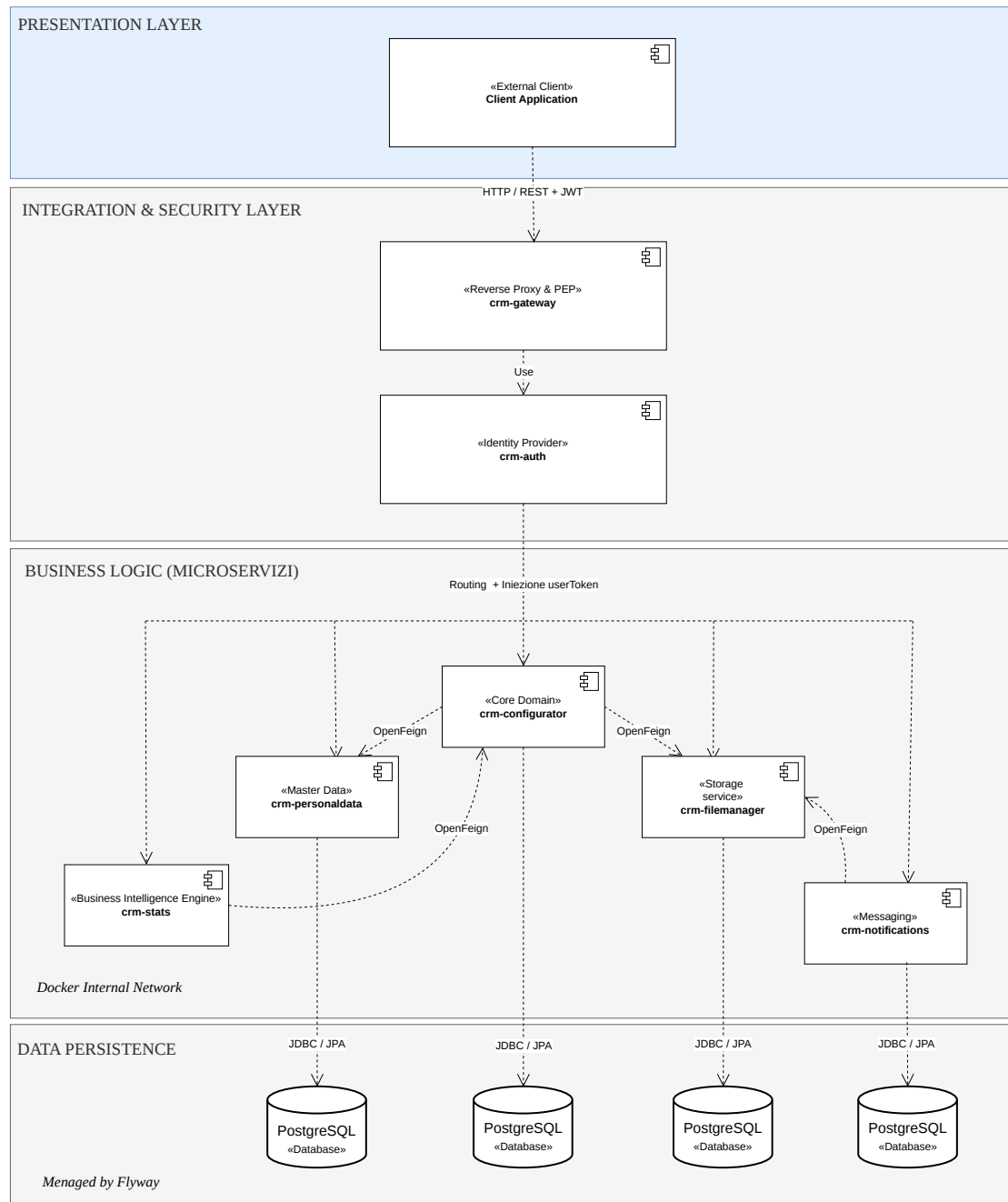


Figura 3.1: Rappresentazione ad alto livello dell'architettura a microservizi della piattaforma CRM sviluppata.

Si parte dall'**API Gateway** (`crm-gateway`), l'unico punto di ingresso (*Single Point of Entry*) per tutto il traffico API proveniente dall'esterno: smista le chiamate verso il microservizio corretto e filtra a monte le richieste non autenticate, per far sì che i moduli interni non debbano occuparsene ciascuno per conto proprio. Subito dietro c'è il **modulo Authentication** (`crm-auth`), che gestisce le utenze di sistema, verifica le credenziali e rilascia i token crittografici usati per l'autenticazione *stateless* dei client.

Il **modulo Personal Data** (`crm-personaldata`) è l'anagrafica centralizzata del CRM, il punto in cui vive tutto ciò che riguarda l'identità dei clienti: si occupa del censimento e della gestione del ciclo di vita dei Clienti (Aziende), dei rispettivi Referenti (persone fisiche) e dei relativi canali, fisici (Indirizzi) e telematici (Contatti). Gli interventi di reingegnerizzazione condotti su questo modulo sono descritti nel dettaglio nella Sezione 3.4.

Il **modulo Configurator** (`crm-configurator`) è il cuore operativo delle vendite: qui vengono gestiti il catalogo prodotti e l'avanzamento della pipeline commerciale (*Opportunity*), modellata come una macchina a stati finiti e rappresentata visivamente tramite una *Kanban Board*, come approfondito nella Sezione 3.5. Al suo fianco lavora il **modulo Stats** (`crm-stats`), il microservizio di Business Intelligence pensato per estrarre e aggregare i KPI di vendita più rilevanti (*Win Rate*, *Sales Trend*, tempi medi di stazionamento in pipeline) e restituirli in una forma utilizzabile (Sezione 3.6).

A completare l'elenco ci sono i **moduli satelliti** `crm-notifications` e `crm-filemanager`: più "di supporto" ma ugualmente essenziali al funzionamento del sistema. Il primo centralizza l'invio delle e-mail transazionali aziendali; il secondo gestisce in isolamento l'upload e la decodifica dei documenti binari (preventivi e contratti in PDF), restituendo al CRM solo i metadati e un puntatore UUID univoco (`file_id`), così da non accrescere inutilmente le dimensioni del database relazionale con dati che non gli competono.

3.3.2 Comunicazione Inter-Servizio: OpenFeign e Configurazione Dinamica

Le chiamate dirette tra un microservizio e l'altro (ad esempio quando `crm-configurator` interroga `crm-personaldata`) si affidano ai client dichiarativi di **Spring Cloud OpenFeign** [7], secondo il pattern generale già descritto nella Sezione 2.2.2. Concretamente, gli endpoint Feign del progetto sono configurati con una proprietà dinamica che prevede un valore di ripiego:

```
feign.client.personaldata.url=  
  ${FEIGN_URL_PERSONALDATA:http://localhost:8081}
```

In locale, quindi, la proprietà punta a `localhost`; nei container Docker la variabile d'ambiente viene sovrascritta con l'alias del container all'interno della rete virtuale (`http://crm-personaldata:8080`). La gestione dinamica dell'URL tramite variabili d'ambiente garantisce la portabilità del servizio, evitando fallimenti di risoluzione di rete durante l'orchestrazione dei container.

3.4 Reingegnerizzazione dei Moduli Legacy

La prima parte del lavoro è stata dedicata alla ristrutturazione di tre microservizi ereditati da un progetto già esistente in azienda.

3.4.1 Modulo `crm-personaldata` (Anagrafica Centralizzata)

Le criticità di dominio appena descritte (Sezione 3.1) si accompagnavano, a livello di codice, a discrepanze piuttosto gravi tra lo schema DDL, le entità JPA e i DTO, del tutto comprensibile, visto che il modulo `personaldata` è il punto d'accesso centrale per le anagrafiche ed è quindi il primo a risentire di ogni disallineamento.

La Tabella 3.3 riassume i principali interventi di refactoring effettuati. Il diagramma E-R in Figura 3.2 mostra come la base dati sia stata riprogettata dal modello originario.

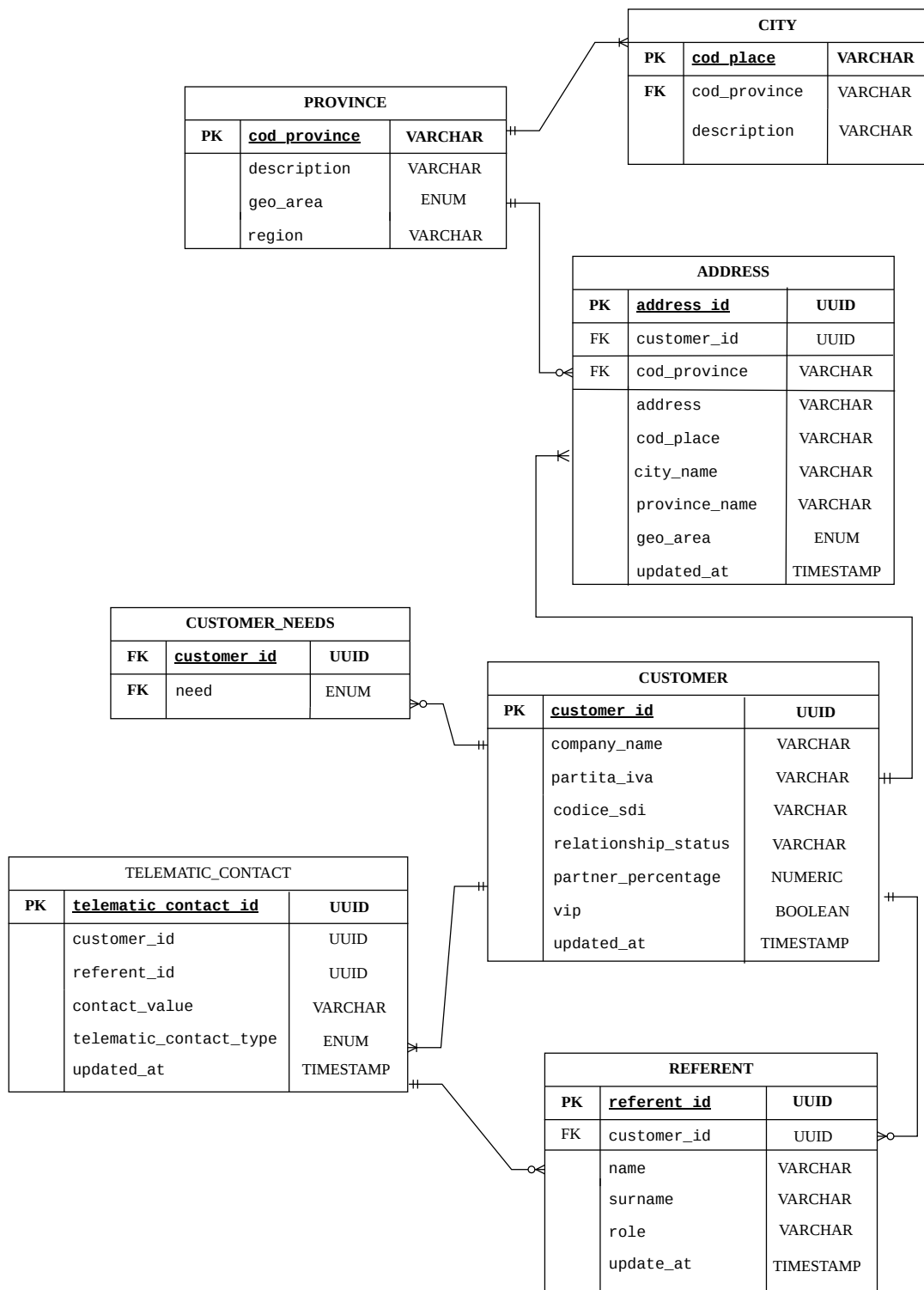


Figura 3.2: Schema E-R Logico-Fisico del modulo `crm-personaldata` a seguito della reingegnerizzazione.

Tabella 3.3: Interventi principali di refactoring sul modulo `crm-personaldata`.

Criticità Identificata	Causa Tecnico/Ingegneristica	Soluzione Applicata
Mismatch nei nomi dei campi	<code>CustomerRequest.contacts</code> non mappava su <code>CustomerModel.addresses</code> in <code>ModelMapper STRICT</code> .	Rinominato il campo in <code>addresses</code> ed eliminati i valori <code>null</code> silenziosi.
Incoerenza su Enum e Tabelle	<code>CustomerEntity</code> usava un Enum singolo, mentre il DDL prevedeva la tabella <code>junction customer_needs</code> .	Sostituito l'Enum con <code>@ElementCollection</code> su <code>Set<NeedType></code> , abilitando il modello multi-need.
Concorrenza non gestita	Rischio di sovrascrittura accidentale di dati in caso di salvataggi simultanei.	Implementato l' Optimistic Locking controllando il <code>timestamp updatedAt</code> in <code>PUT/PATCH</code> .
Mancanza dati fiscali	Assenza di campi obbligatori da specifica (Partita IVA e Codice SDI).	Estesi DDL, Entity, Model, DTO e contratti API in tutti i layer software.

L'inadeguatezza della struttura *legacy* è emersa con evidenza durante la modellazione dei profili aziendali. Nel codice ereditato, la tipologia di relazione con il cliente era modellata tramite un enumeratore scalare rigido (`relationshipStatus`): questo vincolo rendeva impossibile rappresentare entità ibride, come un'azienda che operava simultaneamente sia in veste di partner commerciale sia come committente di servizi di sviluppo o consulenza. Di riflesso, il modulo `crm-configurator` era costretto a forzare transizioni fittizie lungo l'intero ciclo di vita dello stato per ricavare ruoli accessori.

A questa limitazione concettuale si sommava una fragilità a livello di integrazione distribuita: l'aggiunta di un nuovo valore all'Enum nel database o nel servizio fornitore provocava il fallimento della deserializzazione JSON a *runtime* nei client `OpenFeign` che non avevano ancora aggiornato il modello locale. La sostituzione dell'Enum con una collezione di tag persistita tramite `@ElementCollection` su

una struttura `Set<NeedType>` ha risolto entrambi i fronti: sul piano di business ha abilitato l'assegnazione simultanea di molteplici etichette (`SOFTWARE`, `CONSULENZA`, `PARTNER`), mentre sul piano di rete ha reso il contratto REST resiliente all'estensione incrementale dei tipi di relazione.

3.4.2 Modulo *crm-notification*

Incaricato dell'invio automatizzato di e-mail e comunicazioni PEC, questo microservizio è stato ripristinato e disaccoppiato dalle logiche dirette, allineato alle librerie aziendali di sicurezza e migrato integralmente allo standard Jakarta EE.

3.4.3 Modulo *crm-filemanager*

Salvare direttamente file binari (BLOB) nei database relazionali li appesantisce senza motivo, quindi l'archiviazione della documentazione contrattuale è stata confinata nel modulo *crm-filemanager*. Il CRM, di conseguenza, memorizza sul proprio database soltanto i metadati essenziali e un puntatore univoco di tipo **UUID** (`file_id`) al file vero e proprio, il che ottimizza le risorse di memoria e velocizza sensibilmente le query di lettura.

Conclusa la fase di reingegnerizzazione dei moduli *legacy*, la seconda parte del capitolo descrive la progettazione e lo sviluppo *ex-novo* dei due microservizi pensati per completare la piattaforma CRM.

3.5 Sviluppo del Modulo *crm-configurator*

Il microservizio *crm-configurator* gestisce il ciclo di vita delle trattative commerciali, mettendo in relazione catalogo prodotti, anagrafiche clienti e operato dei commerciali.

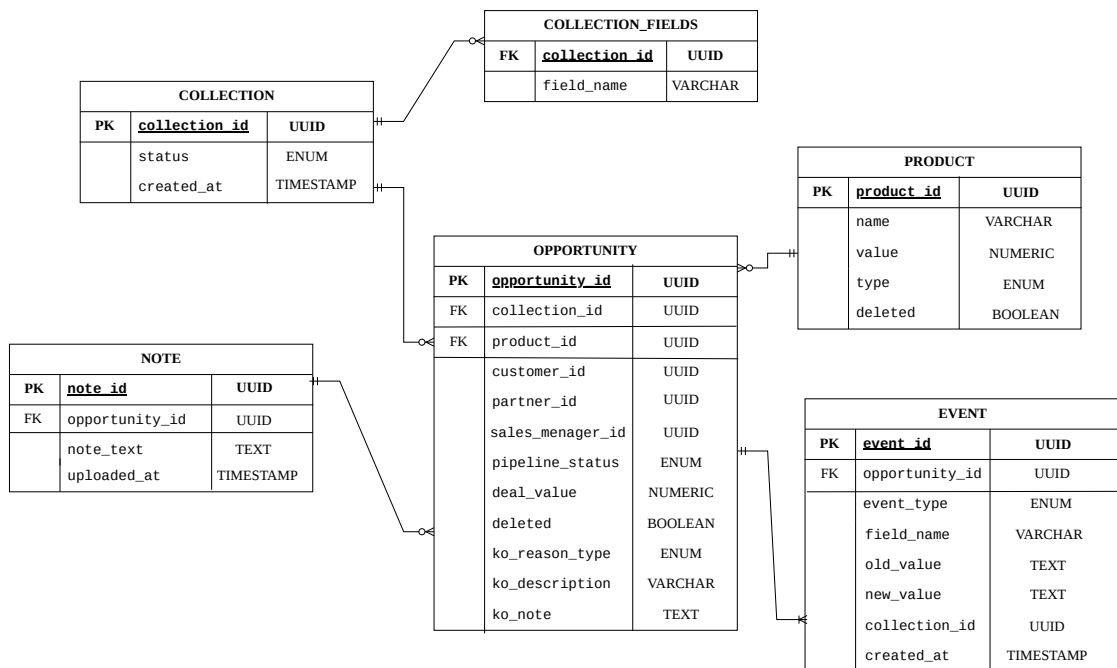


Figura 3.3: Schema E-R Logico-Fisico del modulo *crm-configurator*: architettura relazionale per la gestione delle trattative, catalogo prodotti e Audit Log.

L'architettura dei dati (Figura 3.3) riflette la struttura a microservizi del sistema: le entità esterne come *Clienti* e *Sales Manager* sono referenziate unicamente tramite identificativi *UUID*, rispettando il pattern *Database-per-Service* e demandando l'arricchimento anagrafico a chiamate di rete tramite *OpenFeign*. Lo schema evidenzia inoltre l'adozione del *Soft Delete* per preservare la storicità commerciale (`deleted = true`) e la gestione obbligatoria degli storici operativi (*EVENT* e *NOTE*).

3.5.1 Macchina a Stati e Logiche di Business (Opportunity)

L'entità attorno a cui ruota il modulo è l'Opportunità Commerciale (*Opportunity*). Per modellare il processo di vendita è stata implementata una macchina a stati finiti gestita tramite l'enumeratore *PipelineStatus*, le cui transizioni accompagnano la vendita dall'apertura (*NUOVO*) fino all'esito finale, positivo (*CHIUSO_VINTO*) o negativo (*KO*).

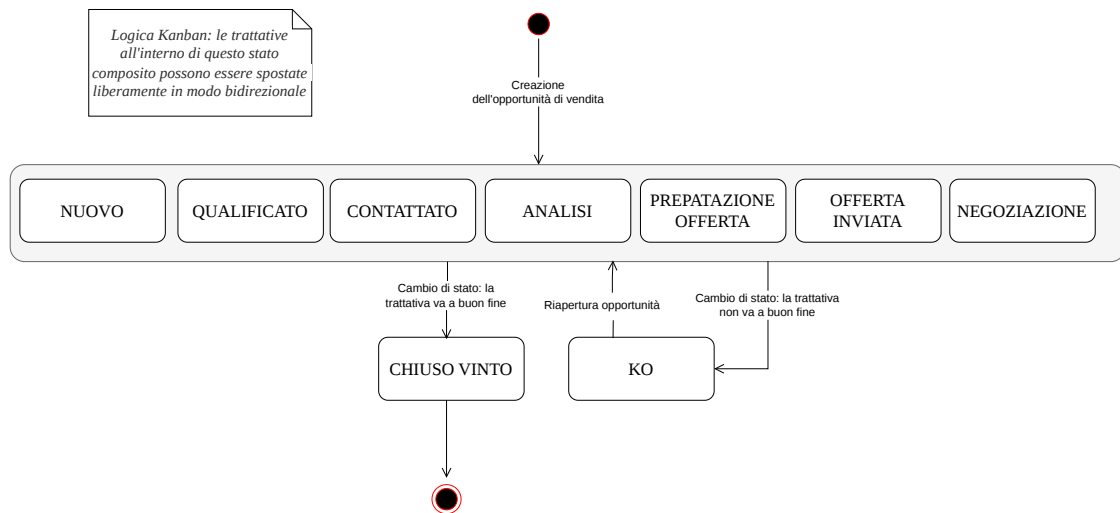


Figura 3.4: Statechart Diagram: Macchina a stati finiti della trattativa commerciale per la logica Kanban.

Lo Statechart Diagram in Figura 3.4 mostra come, per assecondare la flessibilità tipica di una *Kanban Board*, gli stati intermedi siano stati racchiusi in uno *Stato Composito*, al cui interno le transizioni sono libere. Le restrizioni sono state invece applicate soltanto alle transizioni in uscita verso gli stati terminali.

Per proteggere l'integrità del database, sono stati inseriti vincoli rigidi direttamente nel *Service Layer*. Quando l'interfaccia richiede lo spostamento di una trattativa in stato KO (tramite HTTP PATCH), scatta un controllo *fail-fast* sulla presenza della motivazione obbligatoria (`KoReasonType`): l'assenza del campo blocca immediatamente l'operazione, restituendo un codice HTTP 400 (*Bad Request*).

Se in seguito la trattativa viene riaperta o spostata in una fase attiva, il Service intercetta il cambio di stato ed esegue una bonifica automatica dei campi residui legati al fallimento precedente (`koReasonType`, `koDescription`, `koNote`), impedendo così che un'opportunità attiva mantenga a database scorie informative non più coerenti.

Questa rigorosa logica di *backend* si riflette direttamente sull'interfaccia utente frontale, dove ogni colonna della Kanban board corrisponde a uno stato specifico della pipeline (Figura 3.5).

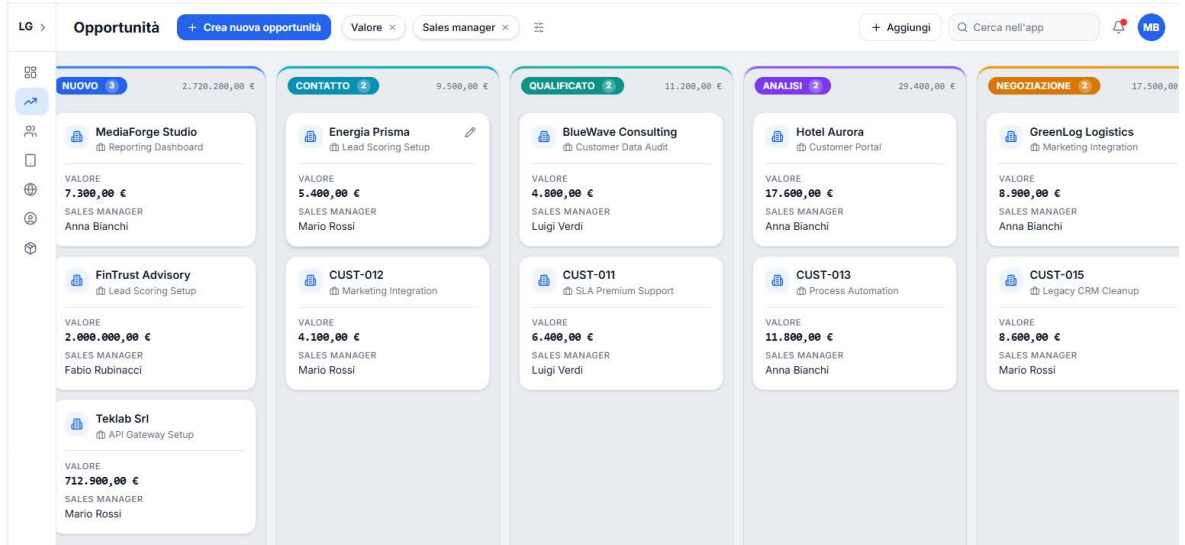


Figura 3.5: Interfaccia utente: Struttura della *Kanban Board* mappata sugli stati della pipeline commerciale.

3.5.2 CQRS, Motore di Ricerca Dinamico e Paginazione

Per consentire al *frontend* di filtrare le trattative senza sovraccaricare la memoria del server, è stato implementato un motore di ricerca *server-side* basato sulle **JPA Criteria API**. Questa scelta rispetta il principio di Separazione delle Responsabilità (CQRS) [12]: le letture complesse sono isolate in un servizio dedicato (*OpportunityFilterService*), separato dalla gestione delle scritture transazionali.

Invece di adoperare query SQL native o clausole JPQL statiche, il motore compone la query a *runtime* sulla base dei soli parametri effettivamente valorizzati nel filtro inviato dal client. L'elaborazione applica sempre la paginazione *server-side* tramite le clausole `LIMIT` e `OFFSET`, incapsulando risultati e metadati di navigazione in un modello di paginazione standardizzato.

3.5.3 Tracciabilità Storica (Event Sourcing)

Ogni modifica apportata a un'opportunità attiva innesca in modo trasparente un meccanismo di *Audit Logging*. La classe entità `Event` registra in maniera immutabile tutti i cambi di stato o di valore economico, memorizzando l'utente responsabile, il timestamp preciso e lo stato precedente e successivo.

Questo tracciamento garantisce la tracciabilità delle attività del team commerciale e fornisce lo storico dati necessario alle elaborazioni analitiche del modulo `stats`.

3.6 Sviluppo del Modulo *crm-stats* (Business Intelligence)

A completamento dell'architettura è stato sviluppato *crm-stats*, un microservizio concepito esclusivamente per l'analisi dei dati. Isolarlo impedisce che l'elaborazione di metriche complesse vada a rallentare le operazioni di vendita quotidiane.

3.6.1 Integrazione OpenFeign e Propagazione della Sicurezza

Poiché il modulo non possiede un proprio database, ricava le informazioni necessarie interrogando il modulo `configurator` tramite chiamate HTTP REST interne gestite da **Spring Cloud OpenFeign**. Proprio durante questa fase di integrazione è emersa un'anomalia di sicurezza legata alla propagazione del token JWT tra i due servizi che verrà discussa nel Capitolo 4.

3.6.2 Algoritmi Analitici di Business Intelligence

L'area di *Business Intelligence* si fonda sull'elaborazione analitica dei flussi transazionali registrati dal CRM.

Sul fronte delle metriche di rendimento economico, il modulo elabora congiuntamente il volume di fatturato mensile (*Sales Trend*) e l'efficacia commerciale complessiva tramite il calcolo del *Win Rate*. Quest'ultimo isola le sole trattative giunte a conclusione formale (escludendo quelle ancora aperte o in lavorazione), quantificando la percentuale di successo secondo la relazione:

$$\text{Win Rate (\%)} = \left(\frac{\text{Opportunità CHIUSO_VINTO}}{\text{Opportunità CHIUSO_VINTO} + \text{Opportunità KO}} \right) \times 100$$

In modo analogo, per l'analisi qualitativa degli insuccessi (*KO Reason Distribution*), l'aggregazione statistica raggruppa le trattative non concluse sulla base dell'enumeratore `KoReasonType` (quali ad esempio `PRICE` o `COMPETITOR`), determinando il peso percentuale di ciascuna motivazione sul totale delle opportunità perse.

La componente computazionale più articolata riguarda tuttavia la misurazione dei tempi di attraversamento del processo (**Tempo Medio per Fase**). A differenza dei conteggi istantanei, questo algoritmo richiede la ricostruzione della cronologia operativa: il servizio interroga il microservizio `crm-configurator` per acquisire la sequenza completa delle transizioni registrate dall'entità `Event`. Una volta ordinati temporalmente gli eventi per ciascuna opportunità, la procedura calcola per ogni intervallo consecutivo la durata di permanenza:

$$\Delta t_i = t_i - t_{i-1}$$

dove t_i e t_{i-1} rappresentano rispettivamente i timestamp di uscita e di ingresso nello stadio considerato. I delta temporali vengono infine aggregati e normalizzati su base giornaliera per ogni singola fase della pipeline, fornendo al management la visibilità analitica necessaria per individuare colli di bottiglia nei flussi di vendita.

Validazione del Sistema, Testing e Sviluppi Futuri

Verificare l'affidabilità del sistema e tutelare l'integrità dei dati sono stati passaggi fondamentali per il rilascio della piattaforma CRM in ambiente di produzione. Il capitolo documenta la strategia di validazione applicata al CRM attraverso test e collaudo concepiti come parte integrante del ciclo di vita del software secondo i principi dell'Ingegneria del Software [3].

L'attività di verifica si è articolata in due fasi: la *Validazione Interna*, con test tecnici delle API REST e troubleshooting dell'infrastruttura di persistenza distribuita, e la *Validazione Esterna*, condotta tramite lo *User Acceptance Testing* (UAT) svolto insieme agli *stakeholder* aziendali [13]. Il capitolo si conclude delineando i possibili sviluppi futuri della piattaforma.

4.1 Fondamenti Teorici del Testing e Strategia Adottata

Seguendo la tassonomia dell'Ingegneria del Software, l'attività di collaudo ha operato su tre livelli [3]: l'individuazione del difetto nel codice (**Fault**), l'intercettazione dello stato interno anomalo (**Error**) e la prevenzione del malfunzionamento percepito dall'utente (**Failure**). Su queste basi, il piano di test ha integrato sia la *Verification* della conformità alle specifiche tecniche sia la *Validation* rispetto ai reali bisogni operativi espressi dagli *stakeholder*.

Trattandosi di un backend *headless*, la strategia di test si è concentrata sul **Black-Box Testing**: i casi di test sono stati definiti validando i *payload* JSON, gli *status code* HTTP e i contratti DTO, indipendentemente dall'implementazione interna dei componenti [3]. Le chiamate HTTP e l'automazione dei flussi di test sulle API REST sono state realizzate con **Postman**.

4.2 Validazione Interna

Il collaudo interno ha verificato il rispetto dei requisiti funzionali e di vincoli non funzionali quali sicurezza e controllo degli accessi tramite RBAC, gestione della concorrenza, validazione degli input e isolamento transazionale [3]. Per ogni endpoint sono stati simulati flussi di dati reali, esaminando sia i percorsi ordinari di esecuzione (*Happy Path*) sia gli scenari d'errore e i casi limite.

La formattazione degli errori è stata demandata al componente aziendale `GlobalExceptionHandler`, che converte le eccezioni di runtime in risposte JSON strutturate sul modello `BaseResponse` (`success: false, message: '...'`).

4.3 Troubleshooting Ingegneristico e Risoluzione delle Criticità

Le sessioni di test e integrazione tra microservizi hanno fatto emergere anomalie architetturali importanti, scaturite dall'interazione tra il layer di persistenza Hibernate, la mappatura dei dati con MapStruct, i meccanismi di sicurezza e la logica delle transizioni di stato.

Tabella 4.1: Principali criticità emerse in fase di troubleshooting e relative soluzioni.

Problema Ricontrato	Causa Tecnica	Soluzione Applicata
Cancellazione di entità figlie inefficace: l'API rispondeva HTTP 200 OK ma il record restava a database [9].	Blocco tra Hibernate e MapStruct [8]: le letture <i>Lazy</i> generate dal mapper per costruire il DTO di risposta ricaricavano l'entità Padre, ancora legata al Figlio nella transazione aperta, inducendo Hibernate a ripristinarlo con un <i>INSERT</i> prima del commit.	Chiamata esplicita a <code>entityManager.flush()</code> subito dopo la <code>delete()</code> e prima dell'invocazione del mapper.
Le chiamate di <code>crm-stats</code> verso <code>crm-configurator</code> via OpenFeign venivano bloccate con HTTP 403 Forbidden [7].	Il token JWT del client si fermava al primo microservizio e non veniva propagato nella chiamata HTTP interna generata da Feign.	Sviluppato un <code>RequestInterceptor</code> Feign che preleva il token dal contesto di sicurezza e lo inietta automaticamente nell'header <code>Authorization</code> di ogni chiamata uscente.

La prima anomalia in tabella evidenzia una criticità tipica dell'integrazione tra ORM e librerie di mapping: l'accesso ai campi in modalità *lazy loading* durante la conversione in DTO altera lo stato del *Persistence Context* di Hibernate prima del *commit* transazionale, rendendo necessario forzare la sincronizzazione tramite `flush()`.

4.4 Validazione Esterna e User Acceptance Testing (UAT)

Una volta stabilizzata la versione funzionale dell'MVP, secondo l'approccio *Lean/Agile* il sistema è passato alla fase di validazione esterna con il committente [2].

L'infrastruttura CRM è stata presentata ufficialmente il 16 luglio 2026 alla *Project Manager*, referente per l'accettazione del prodotto (UAT). Alla sessione ha preso parte anche la direzione aziendale per la valutazione degli aspetti commerciali. La dimostrazione è stata condotta insieme allo sviluppatore *frontend*: la presentazione della logica di business e delle API backend è stata affiancata all'interfaccia utente per validare contestualmente i flussi operativi *end-to-end*.

L'esito della dimostrazione è stato positivo, con poche osservazioni aggiuntive rispetto a quanto già validato precedentemente. Le sessioni di collaudo e accettazione hanno confermato la rispondenza della piattaforma ai requisiti di business concordati. In particolare, il passaggio all'archivio relazionale centralizzato governato dal modello dei *Needs* e l'integrazione di un sistema di audit log immutabile hanno garantito, rispettivamente, l'azzeramento dei record duplicati e la completa tracciabilità storica di ogni transizione di stato.

L'impatto più rilevante per i processi aziendali si è concretizzato nell'area direzionale: il **supporto analitico alle decisioni (BI)**. L'elaborazione automatica e in tempo reale dei KPI strategici (*Win Rate*, *Sales Trend* e tempi medi di attraversamento della pipeline) ha sostituito l'onere della reportistica manuale, fornendo al management una visibilità quantitativa immediata sull'andamento delle vendite.

Sulla base dei riscontri positivi raccolti con la direzione aziendale, durante l'incontro è stata inoltre delineata la prima evoluzione funzionale del prodotto: l'estensione verso un modulo di fatturazione, approfondita nella Sezione 4.5. Il superamento della fase di UAT ha formalizzato il completamento dell'MVP, autorizzandone il rilascio verso l'ambiente di produzione.

4.5 Sviluppi Futuri ed Evoluzione del Prodotto

I riscontri della fase di collaudo hanno confermato la tenuta pratica della modularità adottata: le criticità risolte in fase di *troubleshooting* (Sezione 4.3) sono state circoscritte ai singoli componenti senza impatti a catena sul resto dell'infrastruttura. Analogamente, la separazione tra layer di persistenza e modelli DTO consente di inserire le nuove funzionalità previste senza alterare la logica di business già validata.

In accordo con la direzione aziendale di Large Systems, la roadmap evolutiva della piattaforma individua delle estensioni architetturali volte a completare l'automazione dei processi aziendali.

La priorità a breve termine risiede nell'integrazione del modulo di fatturazione (`crm-billing`). Dal punto di vista del dominio applicativo, la naturale conclusione del ciclo di vita di un'opportunità commerciale giunta allo stato `CHIUSO_VINTO` coincide con la sua formalizzazione amministrativa. L'introduzione di questo nuovo microservizio è concepita per automatizzare la trasformazione della trattativa in documenti contrattuali e preventivi in formato PDF, evitando il reinserimento manuale dei dati contabili ed economici. A livello architetturale, `crm-billing` costituirà il punto di integrazione verso l'esterno per la conformità fiscale, esponendo adapter dedicati per l'interfacciamento con il Sistema di Interscambio (SDI) e abilitando la gestione nativa della fatturazione elettronica secondo gli standard normativi vigenti.

Parallelamente, sono pianificati interventi a supporto della fruizione analitica e dell'operatività del team commerciale. Da un lato, il layer frontend verrà arricchito integrando librerie di *Data Visualization* agganciate direttamente agli endpoint esposti da `crm-stats`, così da tradurre i KPI di Business Intelligence in cruscotti grafici interattivi e navigabili per il management. Dall'altro, è previsto il potenziamento di `crm-notifications` per la gestione proattiva delle scadenze (*Late Items*): il servizio scheduler opererà in background per rilevare anomalie temporali o trattative stagnanti rispetto ai tempi medi di fase, inoltrando notifiche asincrone via e-mail e canali dedicati (come Slack) ai singoli *Sales Manager*.

CAPITOLO 5

Conclusioni

Il percorso di sviluppo ha consentito di trasformare la gestione iniziale delle anagrafiche e delle trattative, originariamente frammentata tra fogli *Excel* ed e-mail, in una piattaforma a microservizi indipendenti, pensata per essere estesa e mantenuta nel tempo.

5.1 Bilancio del Progetto e Obiettivi Raggiunti

L'adozione di un approccio *Agile* basato su un *Minimum Viable Product* ha permesso di rispettare le tempistiche del tirocinio attraverso revisioni periodiche del codice e la validazione progressiva delle funzionalità con la *Project Manager*.

La validazione del sistema è arrivata nella fase di *User Acceptance Testing* con l'approvazione della *Project Manager*. Attualmente la piattaforma consente al team di vendita di gestire l'anagrafica in modo flessibile usando etichette (come "Cliente" o "Partner") che possono coesistere sulla stessa azienda, evitando duplicazioni nei

dati. Allo stesso tempo, la registrazione dello storico delle trattative garantisce la continuità informativa anche in caso di turnover del personale. A questo si aggiunge un ulteriore beneficio gestionale: la direzione può oggi disporre di indicatori costantemente aggiornati, primo fra tutti il *Win Rate*, ricalcolati dal microservizio *Stats* ad ogni cambio di stato o chiusura delle trattative per non gravare sulle prestazioni delle operazioni commerciali quotidiane.

5.2 Valutazione Critica e Competenze Acquisite

Oltre agli aspetti tecnici, questa esperienza ha richiesto lo sviluppo di competenze metodologiche e relazionali, maturate nel confronto quotidiano con il resto del team. Il confronto costante con la *Project Manager* è servito a sviluppare la capacità di tradurre le esigenze di business in specifiche tecniche, ad esempio nella definizione delle etichette anagrafiche o dei criteri di calcolo del *Win Rate*, e di motivare chiaramente le scelte architetturali adottate. È stato altrettanto formativo lavorare con lo sviluppatore *frontend* nelle fasi di integrazione, definendo insieme il formato delle risposte API e la gestione degli errori restituiti dal backend.

La scelta di superare la rigidità dell'enumeratore a favore di un modello anagrafico a tag flessibili (*Needs*) ha dimostrato sul campo come un'analisi approfondita dei requisiti di dominio condizioni direttamente la scalabilità e la manutenibilità dell'intero sistema a lungo termine.

A livello applicativo, le problematiche più stimolanti hanno richiesto un'indagine approfondita sui meccanismi interni del framework. Garantire l'interoperabilità sicura tra microservizi tramite client *OpenFeign* ha imposto la gestione rigorosa della propagazione del contesto di sicurezza e delle credenziali lungo l'intera catena di invocazione *stateless*. L'interazione con il layer di persistenza ha mostrato invece le insidie legate all'astrazione dell'ORM: l'anomalia di cancellazione analizzata nel Capitolo 4 ha evidenziato come l'utilizzo di *Hibernate* non possa prescindere da una comprensione dettagliata del ciclo di vita delle entità gestite, del contesto di persistenza e dei confini transazionali.

Infine, la dimensione infrastrutturale del progetto ha sottolineato l'importanza della coerenza tra gli ambienti. L'adozione sistematica della containerizzazione e degli strumenti di versionamento del database si è rivelata determinante per superare le discrepanze di configurazione tra la fase di sviluppo locale e il contesto di rilascio, confermando come la solidità di un'architettura software dipenda in egual misura dalla qualità del codice e dalla riproducibilità dei suoi processi di deployment.

Bibliografia

- [1] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional, 2015. (Citato a pagina 2)
- [2] E. Ries, *The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses*. Crown Books, 2011. (Citato alle pagine 3 e 36)
- [3] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015. (Citato alle pagine 5, 8, 17, 33 e 34)
- [4] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Inc., 2015. (Citato alle pagine 6, 11 e 17)
- [5] B. Bruegge and A. H. Dutoit, *Object-Oriented Software Engineering Using UML, Patterns, and Java*, 3rd ed. Pearson, 2009. (Citato alle pagine 6, 7 e 20)
- [6] VMware, Inc., "Spring boot official documentation," <https://spring.io/projects/spring-boot>, ultimo accesso: 13 Agosto 2026. (Citato alle pagine 7 e 12)
- [7] —, "Spring cloud openfeign," <https://spring.io/projects/spring-cloud-openfeign>, ultimo accesso: 21 Agosto 2026. (Citato alle pagine 7, 24 e 35)

- [8] MapStruct Authors, "Mapstruct - java bean mappings, the easy way!" <https://mapstruct.org/>, ultimo accesso: 21 Agosto 2026. (Citato alle pagine 9 e 35)
- [9] The PostgreSQL Global Development Group, "Postgresql: The world's most advanced open source relational database," <https://www.postgresql.org/>, ultimo accesso: 18 Agosto 2026. (Citato alle pagine 11 e 35)
- [10] Redgate Software Ltd, "Flyway by redgate - database migration," <https://flywaydb.org/>, ultimo accesso: 18 Agosto 2026. (Citato a pagina 11)
- [11] Docker Inc., "Docker: Accelerated container application development," <https://www.docker.com/>, ultimo accesso: 21 Agosto 2026. (Citato alle pagine 11 e 12)
- [12] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002. (Citato a pagina 30)
- [13] L. Crispin and J. Gregory, *Agile Testing: A Practical Guide for Testers and Agile Teams*. Addison-Wesley Professional, 2009. (Citato a pagina 33)